This is the Definitive Technology Selection & Engineering Rationale for the V1 DTE Platform. As your Principal Software Architect and Enterprise Strategist, I have evaluated the technology landscape strictly against the constraints of our approved architecture, the Ruthless Scope Cut, and our regulatory mandate. The goal is not to chase industry trends; it is to build an incontrovertible, highly maintainable, and operationally simple platform that will reliably serve SMBs and Enterprises in self-hosted environments for the next decade.
Here is the definitive technology stack.

SECTION 1 — Technology Selection Philosophy

The technology evaluation process for this platform is governed by a strict, risk-averse, and longevity-focused philosophy:

- **Evaluation Criteria:** * **Cross-Platform Parity:** Must run identically on Windows Server and Linux (Ubuntu/RHEL).
  - **Ecosystem Maturity:** Must have at least a 10-year proven track record in enterprise financial or regulatory environments.
  - **Operational Simplicity (Self-Hosted):** Must minimize the number of moving parts (e.g., avoiding separate caching clusters or message brokers if the database can handle the load). The fewer services a customer must deploy, the lower the support burden.
  - **Hiring Availability:** Must pull from the largest possible talent pools of pragmatic, enterprise-grade developers.
- **Rejection Criteria:**
  - **Vendor Lock-in:** Any technology that inherently ties the platform to AWS, Azure, or GCP PaaS offerings is explicitly rejected.
  - **Bleeding-Edge / Hype:** Frameworks less than 5 years old, or those undergoing constant paradigm shifts (e.g., the JavaScript framework churn), are rejected.
- **Long-Term Sustainability Criteria:** The language and framework must guarantee backward compatibility, offer Long Term Support (LTS) release cycles, and possess native capabilities for exact-scale financial mathematics and cryptographic operations.

SECTION 2 — Backend Language Selection

The backend language must perfectly support Domain-Driven Design (DDD), Command/Query Responsibility Segregation (CQRS), and strict type safety.

Evaluation

- **TypeScript (Node.js):**
  - *Strengths:* Ubiquity, fast iteration, native JSON handling (great for DTE payloads).
  - *Weaknesses:* Dynamic typing at runtime. The single-threaded event loop can bottleneck during heavy concurrent cryptographic hashing (required for our audit and evidence pipelines). High ecosystem churn.
- **Java:**
  - *Strengths:* Unmatched enterprise longevity, massive talent pool, deep DDD ecosystem.
  - *Weaknesses:* Heavy memory footprint and slow startup times, which penalize small SMBs running the platform on limited VPS infrastructure. Verbosity limits development

speed.

- **Go:**
  - *Strengths:* Compiles to a single binary (amazing for self-hosting), ultra-fast, low memory footprint.
  - *Weaknesses:* The enterprise talent pool is smaller. Its lack of mature DDD/CQRS structural frameworks often leads to reinventing architectural wheels. Verbose error handling can obscure domain logic.
- **C# (.NET 8+):**
  - *Strengths:* First-class cross-platform support (Linux/Windows). Exceptional performance (comparable to Go). Massive enterprise talent pool. Strict, static typing. Perfect alignment with DDD/CQRS architectures. Built-in, high-performance JSON processing.
  - *Weaknesses:* Historical (though factually outdated) perception as a Windows-only ecosystem.

Selection: C# (.NET 8+ LTS)

Rationale: C# and .NET 8 represent the absolute best intersection of performance, maintainability, and enterprise talent availability. It provides the strict runtime type guarantees required for a regulatory financial platform. Crucially, .NET allows us to compile the application as a self-contained executable, meaning customers do not even need to install the .NET runtime on their host servers, radically simplifying self-hosted deployments across both Windows and Linux.

SECTION 3 — Backend Framework Selection

Evaluation

Having selected C#, the leading framework is ASP.NET Core. The decision lies in how we structure the API.

- *Strengths:* Built-in Dependency Injection, world-class middleware pipeline (perfect for JWT auth and global error handling), industry-leading performance.
- *Weaknesses:* Can lead to over-engineering if developers abuse abstractions.
- *Operational Complexity:* Very low. It hosts natively in Kestrel (its own web server) or behind IIS/Nginx.

Selection: ASP.NET Core Web API paired with MediatR

Rationale: We will use ASP.NET Core Web API using Controller architectures, strictly integrating the **MediatR** library. MediatR enforces the CQRS requirement defined in the Application Architecture Blueprint. Controllers will be explicitly stripped of business logic; they will merely route HTTP requests to MediatR Commands (for mutations) and Queries (for reads). This guarantees the strict separation of intent required by our Engineering Standards.

SECTION 4 — Database Selection

The database is the most critical component of the platform. It must enforce the *Snapshot Law* via JSON, provide exact financial precision, and natively support append-only triggers to defend

the audit trail.

Evaluation

- **SQL Server:**
  - *Analysis:* Exceptional engine, but licensing costs are prohibitive for SMB customers. While it runs on Linux, it is heavy and operationally complex.
- **MySQL:**
  - *Analysis:* Ubiquitous and free, but its JSON querying capabilities and advanced constraint systems trail behind competitors.
- **PostgreSQL:**
  - *Analysis:* True open-source (zero licensing costs). Best-in-class JSONB support (critical for our immutable issuer_snapshot and receptor_snapshot). Native table partitioning (vital for 10-year evidence retention). Advanced trigger capabilities to physically enforce immutability on audit tables.

Selection: PostgreSQL (Version 15+)

Rationale: PostgreSQL perfectly maps to the Physical Data Design. It eliminates database licensing from the customer's Total Cost of Ownership (TCO). Its JSONB implementation allows us to store complex DTE snapshots securely while retaining indexable, high-speed querying. The ability to use BYTEA for evidence blobs and NUMERIC(18,4) for exact financial mathematics makes it the only responsible choice.

SECTION 5 — Queue & Background Processing Strategy

Requirement Check: The API Specification mandates that MH transmission is asynchronous. Therefore, a background processing mechanism is required.

Evaluation

- **Redis / BullMQ / RabbitMQ:** Exceptional throughput, but forces the customer to deploy, monitor, and maintain a separate infrastructure component (the message broker). Unacceptable operational overhead for an SMB running a single VPS.
- **.NET Background Services (Channels):** In-memory and fast, but lacks persistence. If the server crashes, queued MH transmissions are lost.
- **Hangfire (backed by PostgreSQL):** A robust background job processor for .NET that uses the primary database as its persistent queue.

Selection: Hangfire (PostgreSQL Storage)

Rationale: Hangfire eliminates the need for a standalone message broker (no Redis, no RabbitMQ). It relies entirely on our existing PostgreSQL database to persist queues, retries, and failure states. This drastically reduces deployment complexity for self-hosted customers while perfectly fulfilling the asynchronous transmission requirements.

SECTION 6 — Evidence Storage Strategy

Evidence (Signed JSON payloads, generated PDFs, MH Acceptance Stamps) must be retained for 10+ years and cryptographically verified.

Evaluation

- **Filesystem Storage:** Saving PDFs/JSONs to a server directory. Highly vulnerable to OS-level deletion, permission misconfigurations, and requires complex backup synchronization (DB and Filesystem must be backed up at the exact same millisecond to prevent corruption).
- **Database Storage:** Storing files as binary blobs within the database rows.

Selection: Database Storage (PostgreSQL BYTEA with TOAST)

Rationale: As dictated by the Physical Data Design, we will store evidence natively in PostgreSQL using BYTEA columns. PostgreSQL automatically manages large blobs via TOAST (The Oversized-Attribute Storage Technique), keeping the primary table fast.
Why? **Transactional Self-Containment.** When a customer backs up their PostgreSQL database, they are backing up the relational data, the audit trail, and the cryptographic evidence simultaneously. A single .bak or .sql file holds the entire legally defensible truth of the corporation.

SECTION 7 — Authentication Strategy

Evaluation

- **Session-based (Stateful):** Requires sticky sessions or a distributed cache (Redis) if the application scales horizontally.
- **Token-based (Stateless JWT):** Self-contained, scalable, and natively understood by mobile apps or ERP integrations.

Selection: Token-Based (JWT) via HTTPOnly Cookies

Rationale: We will issue short-lived JWTs. To protect against XSS attacks in the UI, the JWT will be strictly delivered via HTTPOnly, SameSite=Strict cookies to the browser. This satisfies the Security Architecture, allows for stateless backend scaling, and paves a clean road for V1.5 service account (API key) integration.

SECTION 8 — Deployment Strategy

Customers span from sophisticated Enterprises to local SMBs. The deployment strategy must accommodate diverse operational maturity.

Evaluation

- **Native Deployment:** Customers install the .exe or Linux binary directly, configure systemd/IIS, and manage local databases.
- **Containerized Deployment:** Customers use Docker Compose to spin up the application

and database together.

Selection: Hybrid (Container-First, Native-Supported)

Rationale: 1. **Container-First:** We will ship an official Docker image and a standard docker-compose.yml. This is the gold standard for predictable, OS-agnostic deployments. It bundles the App and Hangfire in one container, alongside the PostgreSQL container.
2. **Native-Supported:** Because we use .NET 8, we will also compile a Self-Contained Native Executable. For Windows Server environments where IT policies prohibit Docker, the customer can simply run the .exe and point it via a connection string to their existing PostgreSQL server.

SECTION 9 — Logging & Observability Stack

Self-hosted software cannot phone home with telemetry (privacy/regulatory violations), nor should it force complex monitoring stacks like ELK onto SMBs.

- **Logging Approach: Serilog** within .NET. We will output structured JSON logs directly to the local filesystem (with daily rolling policies) and to stdout.
- **Metrics & Health:** Expose standard ASP.NET Core Health Checks endpoints (/health, /health/db).
- **Rationale:** We remain purely pragmatic. Enterprise customers will use Datadog, Splunk, or Promtail to scrape the JSON files and /health endpoints. SMB customers can read the text files if things go wrong. We provide the hooks; we do not force the infrastructure.

SECTION 10 — Testing Strategy

The testing stack must guarantee our invariants and audit defensibility.

- **Unit Testing: xUnit** + **FluentAssertions**. The industry standard. Used to test pure domain logic (e.g., verifying DocumentTotals computation).
- **Integration Testing: Testcontainers** for .NET. This is non-negotiable. Integration tests must spin up a real PostgreSQL container inside the CI/CD pipeline. Mocking the database is forbidden, as we rely heavily on physical PostgreSQL triggers to enforce audit immutability.
- **End-to-End Testing: Playwright**. Used to script headless browser tests verifying the manual UI composition workflow from Draft to Validated to Signed.

SECTION 11 — Technology Risk Analysis

- **C# / .NET 8:** * *Risk:* Developers over-engineering abstractions (interfaces for everything).
  - *Mitigation:* Strict code review enforcement governed by the Engineering Standards. Controllers talk only to MediatR; MediatR handlers execute domain logic directly.
- **PostgreSQL (BYTEA bloat):**
  - *Risk:* Storing millions of PDFs in the database will cause the tables to grow massively over 10 years.
  - *Mitigation:* The Physical Data Design already mitigates this via PostgreSQL Table Partitioning (RANGE partitioning by issue_date). Old years can be moved to cheaper tablespaces by customer DBAs without application code changes.
- **Hangfire (Database Polling):**
  - *Risk:* Hangfire polls the database to check for new queued jobs, which can add DB load.

    - *Mitigation:* Easily managed by PostgreSQL at the expected volume of DTEs. The trade-off of removing Redis from the customer's deployment burden heavily outweighs the minimal polling overhead.

SECTION 12 — Final Stack Recommendation

This stack guarantees longevity, strict adherence to the regulatory Domain and Data specifications, and exceptional operational simplicity for our customers.

- **Backend Language:** C# (.NET 8 LTS)
- **Backend Architecture Framework:** ASP.NET Core Web API with MediatR (CQRS enforcement)
- **Primary Database:** PostgreSQL 15+ (Relational, JSONB, BYTEA, Constraints)
- **Background Processing:** Hangfire (PostgreSQL Storage)
- **Authentication:** JWT via HTTPOnly Cookies
- **Evidence Storage:** PostgreSQL BYTEA (TOAST)
- **Deployment:** Docker / Self-Contained Native Executables
- **Observability:** Serilog (Structured JSON files) + ASP.NET Health Checks
- **Testing:** xUnit + Testcontainers + Playwright

Summary of Trade-Offs: We traded the ultra-high queueing throughput of Redis/BullMQ for the operational simplicity of Hangfire/PostgreSQL. We traded the ease of filesystem storage for the strict transactional integrity of Database BYTEA storage. We rejected hyped, bleeding-edge JavaScript frameworks in favor of the boring, predictable, highly-typed, multi-decade stability of C# and PostgreSQL.

This stack will survive the next decade.